

UNIT-II

Process

A process is a program in execution. For example, when we write a program in C or C++ and compile it, the compiler creates binary code. The original code and binary code are both programs. When we actually run the binary code, it becomes a process.

- A process is an 'active' entity instead of a program, which is considered a 'passive' entity.
- A single program can create many processes when run multiple times; for example, when we open a .exe or binary file multiple times, multiple instances begin (multiple processes are created).

How Does a Process Look Like in Memory?

A process in memory is divided into several distinct sections, each serving a different purpose. Here's how a process typically looks in memory.



Process structure

- **Text Section:** A text or code segment contains executable instructions. It is typically a read only section
- **Stack:** The stack contains temporary data, such as function parameters, returns addresses, and local variables.
- **Data Section:** Contains the global variable.
- **Heap Section:** Dynamically memory allocated to process during its run time.

Attributes of a Process

A **process** has several important attributes that help the operating system manage and control it. These attributes are stored in a structure called the Process Control

Block (PCB) (sometimes called a task control block). The PCB keeps all the key information about the process, including:

1. **Process ID (PID):** A unique number assigned to each process so the operating system can identify it.
2. **Process State:** This shows the current status of the process, like whether it is running, waiting, or ready to execute.
3. **Priority and other CPU Scheduling Information:** Data that helps the operating system decide which process should run next, like priority levels and pointers to scheduling queues.
4. **I/O Information:** Information about input/output devices the process is using.
5. **File Descriptors:** Information about open files and network connections.
6. **Accounting Information:** Tracks how long the process has run, the amount of CPU time used, and other resource usage data.
7. **Memory Management Information:** Details about the memory space allocated to the process, including where it is loaded in memory and the structure of its memory layout (stack, heap, etc.).

These attributes in the PCB help the operating system control, schedule, and manage each process effectively.

Operation on The Process

- **Creation:** A process is created and placed in the ready queue (main memory), where it waits to be executed.
- **Scheduling:** The operating system selects one process from the ready queue to run next. This selection step is called scheduling.
- **Execution:** The CPU starts running the chosen process. If the process needs to wait for an event or resource, it becomes blocked, and the CPU switches to another process.
- **Termination:** Once the process completes its task, the OS terminates it and clears its context.
- **Blocking:** When a process waits for an event or resource (like I/O), it enters the blocked state and cannot continue until that event is completed.
- **Context Switching:** When the OS switches from one process to another, it saves the current process's context and loads the next one. This switching is called context switching.
- **Inter-Process Communication (IPC):** Processes often need to exchange data or coordinate. The OS supports IPC using mechanisms like shared memory, message passing, and synchronization tools.

Process Control Block (PCB)

A Process Control Block (PCB) is a data structure used by the operating system to keep track of process information and manage execution.

- Each process is given a unique Process ID (PID) for identification.
- The PCB stores details such as process state, program counter, stack pointer, open files, and scheduling info.
- During a state transition, the OS updates the PCB with the latest execution data.
- It also includes register values, CPU quantum, and process priority.
- The Process Table is an array of PCBs that maintains information for all active processes.

Structure of the Process Control Block

A Process Control Block (PCB) is a data structure used by the operating system to manage information about a process. The process control keeps track of many important pieces of information needed to manage processes efficiently. The diagram helps explain some of these key data items.



- **Pointer:** It is a stack pointer that is required to be saved when the process is switched from one state to another to retain the current position of the process.
- **Process state:** It stores the respective state of the process.
- **Process number:** Every process is assigned a unique id known as process ID or PID which stores the process identifier.
- **Program counter:** Program Counter stores the counter, which contains the address of the next instruction that is to be executed for the process.
- **Register:** Registers in the PCB, it is a data structure. When a process is running and its time slice expires, the current value of process specific registers would be stored in the PCB and the process would be swapped out. When the process is scheduled to be run, the register values are read from the PCB and written to the CPU registers. This is the main purpose of the registers in the PCB.
- **Memory limits:** This field contains the information about memory management system used by the operating system. This may include page tables, segment tables, etc.
- **List of Open files:** This information includes the list of files opened for a process.

The Process Control Block (PCB) is stored in a special part of memory that normal users can't access. This is because it holds important information about the process. Some operating systems place the PCB at the start of the kernel stack for the process, as this is a safe and secure spot.

Process States

Process states represent the dynamic lifecycle of computational execution. Understanding these states and their transitions is crucial for grasping how operating systems manage computational resources efficiently.

a. New State

- Process is being created
- Resources are being allocated
- Not yet ready for execution
- Initialization of process control block (PCB)

b. Ready State

- Process is prepared for execution
- Waiting in the process queue
- All required resources are available
- Awaiting CPU scheduler selection

c. Running State

- Process is currently executing
- Actively using CPU resources
- Instructions are being processed
- Can transition to other states based on events

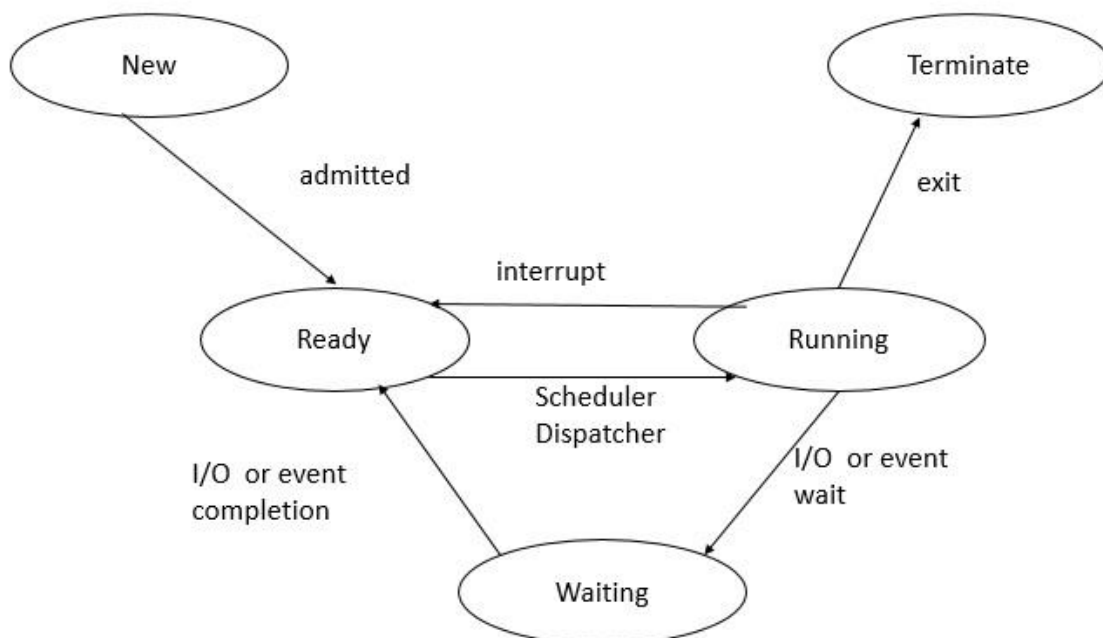
d. Waiting (Blocked) State

- Process is temporarily suspended
- Waiting for external event
- Common triggers:
 - I/O operations
 - Inter-process communication
 - Resource allocation
 - External synchronization signals

e. Terminated State

- Process execution is complete
- Resources are being deallocated
- Process control block is being removed
- Memory is being freed

Now let us see the state diagram of these process states –



State transition diagram

Step 1 – Whenever a new process is created, it is admitted into ready state.

Step 2 – If no other process is present at running state, it is dispatched to running based on scheduler dispatcher.

Step 3 – If any higher priority process is ready, the uncompleted process will be sent to the waiting state from the running state.

Step 4 – Whenever I/O or event is completed the process will send back to ready state based on the **interrupt signal** given by the running state.

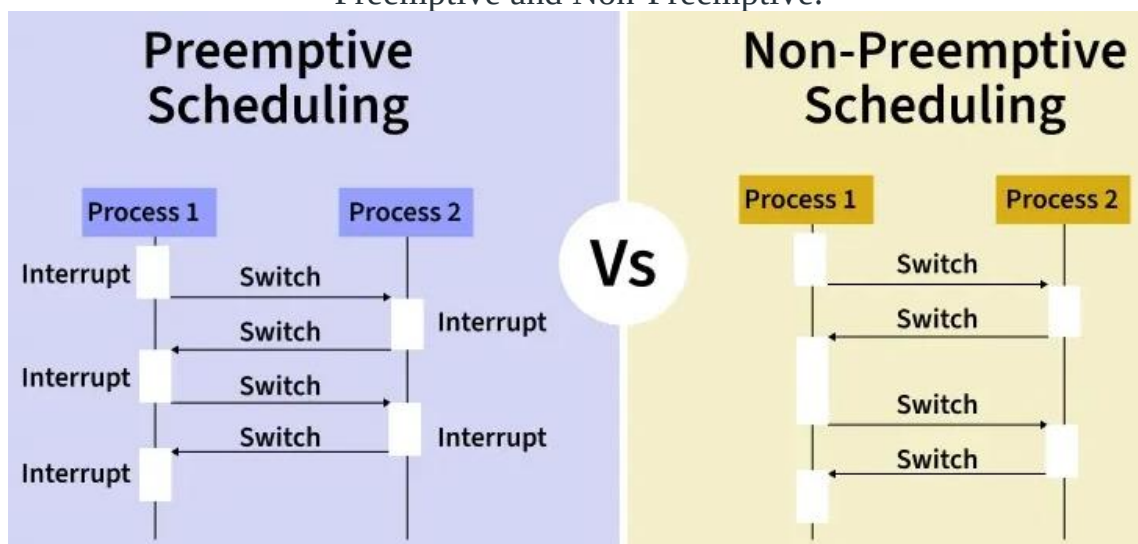
Step 5 – Whenever the execution of a process is completed in running state, it will exit to terminate state, which is the completion of process.

Preemptive and Non-Preemptive Scheduling

CPU scheduling in operating systems is the method of selecting which process in the ready queue will execute on the CPU next. It aims to utilise the processor efficiently while minimising waiting and response times. By determining an optimal execution order, CPU scheduling enhances overall system performance, supports smooth multitasking, and improves the user experience.

Scheduling can be broadly classified into two types:

Preemptive and Non-Preemptive.



Preemptive Scheduling

In Preemptive scheduling, the operating system can interrupt a running process to allocate the CPU to another process usually due to priority rules or time-sharing policies. A process may be moved from Running → Ready state before it finishes. In the following example P2 is preempted at time 1 due to arrival of a higher priority process.

Examples:

- Round Robin
- Shortest Remaining Time First (SRTF)
- Priority Scheduling (Preemptive version)

Process	Arrival Time	CPU Burst Time (in millisec.)
P0	3	2
P1	2	4
P2	0	6
P3	1	4



Preemptive Scheduling

Advantages

- Prevents one process from monopolizing CPU
- Better average response time in multi-user systems
- Widely used in modern OS (Windows, Linux, macOS)

Disadvantages

- More complex to implement
- Higher overhead from context switching
- Can cause starvation of low-priority processes
- Risk of concurrency issues if pre-empted during shared resource access

Non-Preemptive Scheduling

In non-Preemptive scheduling, once a process starts using the CPU, it runs until it finishes or moves to a waiting state. The OS cannot forcibly take away the CPU. Below is the table and Gantt Chart according to the **First Come First Serve (FCFS)** Algorithm: We can notice that every process finishes execution once it gets CPU.

Process	Arrival Time	CPU Burst Time (in millisec.)
P0	3	2
P1	2	4
P2	0	6
P3	1	4



Non-Preemptive Scheduling

Examples:

- First Come First Serve (FCFS)
- Shortest Job First (SJF)
- Priority Scheduling (non-pre-emptive version)

Advantages

- It is easy to implement in an operating system. It was used in Windows 3.11 and early macOS.
- It has a minimal scheduling burden.
- Less computational resources are used.

Disadvantages

- It is open to denial-of-service attack. A malicious process can take CPU forever.
- Since we cannot implement round robin, the average response time becomes less.

Preemptive vs. Non-Preemptive Scheduling

Preemptive Scheduling	Non-Preemptive Scheduling
In these resources (CPU Cycle) are allocated to a process for a limited time.	Once resources (CPU Cycle) are allocated to a process, the process holds it till it completes its burst time or switches to waiting state
Process can be interrupted in between.	Process cannot be interrupted until it terminates itself or its time is up
If a process having high priority frequently arrives in the ready queue, a low priority process may starve	If a process with a long burst time is running CPU, then later coming process with less CPU burst time may starve
It has overheads of scheduling the processes	It does not have overheads
Average process response time is less	Average process response time is high
Decisions are made by the scheduler and are based on priority and time slice allocation	Decisions are made by the process itself and the OS just follows the process's instructions
More as a process might be preempted when it was accessing a shared resource.	Less as a process is never preempted.
Examples of Preemptive scheduling are Round Robin and Shortest Remaining Time First	Examples of Non-Preemptive scheduling are First Come First Serve and Shortest Job First